



Java: Module 3

▼ Exception

▼ Introduction

An exception (or exceptional event) is a problem that arises during the execution of a program. When an **Exception** occurs the normal flow of the program is disrupted and the program/Application terminates abnormally, which is not recommended, therefore, these exceptions are to be handled.

An exception can occur for many different reasons. Following are some scenarios where an exception occurs.

- A user has entered an invalid data.
- A file that needs to be opened cannot be found.
- A network connection has been lost in the middle of communications or the JVM has run out of memory.

▼ Exception Handling Techniques

There are several techniques for handling exceptions in Java:

▼ 1. Try-catch blocks:

Try-catch blocks allow you to "try" a block of code and "catch" any exceptions that are thrown. This is the most common technique for

handling exceptions in Java.

Syntax

```
try {
    // Code that may throw an exception
} catch (ExceptionType1 e) {
    // Code to handle ExceptionType1
} catch (ExceptionType2 e) {
    // Code to handle ExceptionType2
} catch (ExceptionType3 e) {
    // Code to handle ExceptionType3
}
}
```

Example

```
import java.io.FileInputStream;
import java.io.FileNotFoundException;

public class TryCatchExample {
    public static void main(String[] args) {
        try {
            FileInputStream fis = new FileInputStream("myfile.txt");
            // code that might throw an exception goes here
        } catch (FileNotFoundException e) {
            // code to handle the FileNotFoundException goes here
            System.out.println("File not Found");
        }
    }
}

/*This code will output 'File not Found' if in case the myfile.txt is
in the destination*/
```

In this example, the "try" block contains a statement that creates a new `FileInputStream` object. If the file "myfile.txt" does not exist, this will throw a `FileNotFoundException`. The catch block catches this exception and executes the code inside it.

In this case, the catch block simply handles the exception, but you could also use it to log the exception, display an error message to the user, or take other appropriate action.

It's important to note that the catch block must specify the type of exception it is catching. In this example, the catch block catches a `FileNotFoundException`, which is a specific type of exception that is thrown when a file cannot be found. If you want to catch any exception, you can use the general-purpose `Exception` class as the catch parameter.

▼ 2. The "finally" block:

The "finally" block is an optional block of code that can be used in conjunction with a try-catch block. The code in the finally block will always be executed, whether or not an exception is thrown. This can be useful for cleaning up resources or performing other tasks that need to be done regardless of whether an exception occurs.

syntax

```
finally{  
}
```

Example

```
// Java program to demonstrate finally block  
// When exception rise and not handled by catch  
  
import java.io.*;  
  
class GFG {  
    public static void main(String[] args)  
    {  
        try {  
            System.out.println("Inside try block");  
  
            // Throw an Arithmetic exception  
            System.out.println(34 / 0);  
        }  
  
        // Can not accept Arithmetic type exception  
        // Only accept Null Pointer type Exception  
        catch (NullPointerException e) {  
  
            System.out.println(  
                "catch : exception not handled.");  
        }  
  
        // Always execute  
        finally {  
  
            System.out.println(  
                "finally : i will execute always.");  
        }  
        // This will not execute  
        System.out.println("i want to run");  
    }  
}
```

ouput

```
Inside try block  
finally : i will execute always.  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at GFG.main(File.java:10)
```

Here, the program throws an exception but not handled by catch so finally block execute after the try block and after the execution of finally block program terminate abnormally, But finally block execute fine.

▼ 3. The "throws" clause:

If you do not want to handle an exception in a try-catch block, you can use the "throws" clause in the method declaration to specify that the method may throw an exception. This will cause the exception to propagate up the call stack until it is caught and handled by another method or by the top level of the program.

Syntax

```
returnType methodName(parameterList) throws exceptionList {  
    // method body  
}
```

Example

```
import java.io.FileInputStream;  
import java.io.FileNotFoundException;  
  
public class ThrowsExample {  
    public static void main(String[] args) throws FileNotFoundException {  
        FileInputStream fls = new FileInputStream("Myfile.txt");  
    }  
}
```

Here the previous code is converted to enable the throws expression,, whenever the code execute without the `throws FileNotFoundException` it shoots an exception, if there is no file named "Myfile.txt" in the file location . Here the throws clause throws out the exceptional case of File not found.

▼ 4. The "throw" statement:

The "throw" statement allows you to throw an exception manually. This can be useful if you want to throw a custom exception or if you want to throw an exception that has been caught and modified in some way.

```
public class Main {  
    static void checkAge(int age) {  
        if (age < 18) {  
            throw new ArithmeticException("Access denied - You must be at least 18 years old.");  
        }  
    }  
}
```

```

    }
    else {
        System.out.println("Access granted - You are old enough!");
    }
}

public static void main(String[] args) {
    checkAge(15); // Set age to 15 (which is below 18...)
}

```

▼ 5. The "assert" statement:

The "assert" statement can be used to test a boolean expression and throw an `AssertionError` if the expression evaluates to false. This is typically used for debugging and testing purposes.

```

public class AssertDemo {
    public static void main(String[] args) {
        int x = 10;
        assert x > 0; // This assertion will succeed
        System.out.println("x is positive");

        x = -1;
        assert x > 0 : "x is not positive"; // This assertion will fail and throw an AssertionError
        System.out.println("This line will not be reached");
    }
}

```

In this example, the first `assert` statement tests whether `x` is greater than 0, and since it is, the assertion succeeds and the program continues to the next line. The second `assert` statement tests whether `x` is positive, and since it is not, the assertion fails and an `AssertionError` is thrown.

Note that `assert` statements are disabled by default in Java, so they will not be executed unless the `-enableassertions` or `-ea` flag is passed to the `java` command when running the program.

▼ Creating your own exceptions

In Java, you can create your own exception class by extending the `Exception` class or one of its sub-classes. This allows you to define a custom exception type that can be thrown and caught in your program.

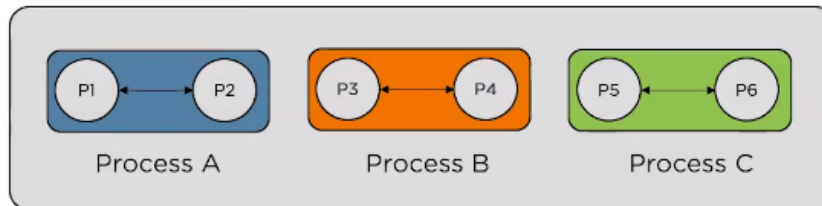
```

public class CustomException extends Exception {
    public CustomException(String message) {
        super(message);
    }
}

```

▼ Threads

▼ What is a Thread?



A Thread is an individual, light-weight, and a smallest unit of a given process. There are multiple threads in a single process and each thread is independent of the other.

Threads are inter dependent, sometimes it is necessary to switch between processes to another or one thread to another .

▼ Multitasking

In Java, multitasking refers to the ability of a central processing unit (CPU), or a single core in a multi-core processor, to execute multiple processes or threads concurrently. This is achieved by allowing each process or thread to run for a short period of time before interrupting it and running the next process or thread. This process is known as **"context switching."**

To create a multi-threaded program in Java, you can use the `Thread` class

Example

```
public class MyThread extends Thread {
    public void run() {
        // code to be executed in the new thread
    }
}

// Create a new thread
MyThread t = new MyThread();

// Start the new thread
t.start();
```

▼ Creation of new Threads

To create a new thread, your program will either extend `Thread` or implement the `Runnable` interface.

By Extending the thread class

Extend the `Thread` class and override the `run()` method. Then, create an instance of the class and call the `start()` method to start the new thread.

```
public class MyThread extends Thread {
    public void run() {
        // code to be executed in the new thread
    }
}

// Create a new thread
MyThread t = new MyThread();

// Start the new thread
t.start();
```

Here, the code creates a new thread by extending the `Thread` class and overriding the `run()` method. The `start()` method is then called to start the new thread, which will execute the code in the `run()` method.

But calling the `start()` method does not actually cause the code in the `run()` method to be executed immediately. Instead, it causes the new thread to be scheduled for execution as soon as possible. The actual execution of the thread depends on the scheduling policies of the operating system and the availability of CPU resources.

By Implementing the Runnable interface

Implement the `Runnable` interface and pass an instance of the class to the `Thread` constructor. Then, call the `start()` method to start the new thread.

```
public class MyRunnable implements Runnable {
    public void run() {
        // code to be executed in the new thread
    }
}

// Create a new runnable
MyRunnable r = new MyRunnable();

// Create a new thread and pass the runnable as an argument
Thread t = new Thread(r);

// Start the new thread
t.start();
```

The code creates a new thread by implementing the `Runnable` interface and passing an object of the class to the `Thread` constructor. The `start()`

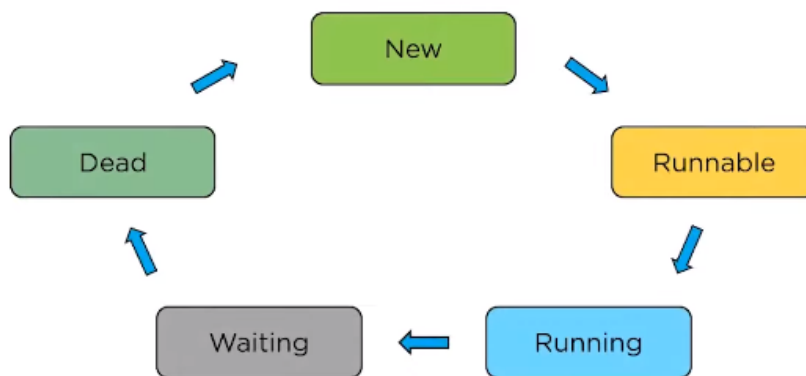
method is then called to start the new thread, which will execute the code in the `run()` method.

But calling the `start()` method does not actually cause the code in the `run()` method to be executed immediately. Instead, it causes the new thread to be scheduled for execution as soon as possible. The actual execution of the thread depends on the scheduling policies of the operating system and the availability of CPU resources.

▼ States of Thread

https://youtu.be/2HN_CDik_s

Thread Life cycle



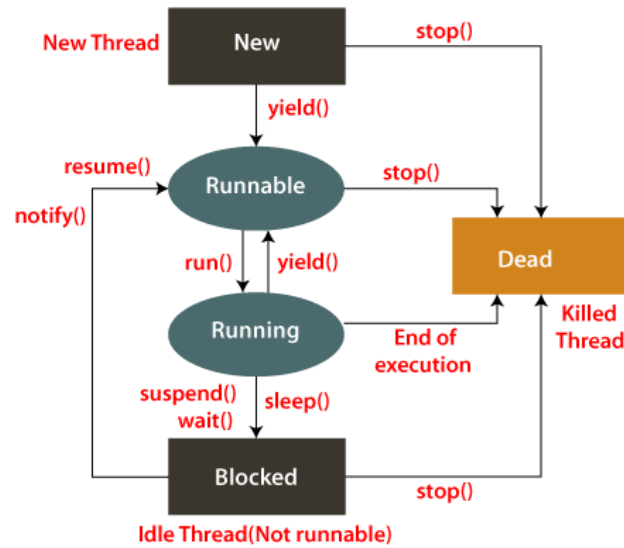
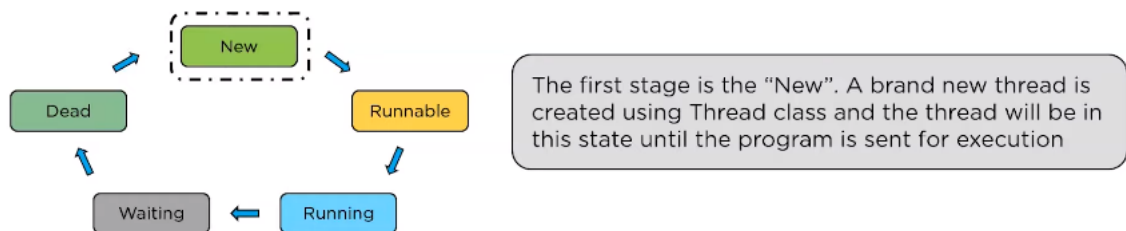


Fig: State Transition Diagram of a Thread

The States of a Thread are:

▼ New (new born state)



In this state, Thread object is created but it cannot execute any program statement because it is not in an execution state of the thread. Only start() method can be called on a new thread; otherwise, an `IllegalThreadStateException` will be thrown.

▼ Runnable



The second phase of a new-born thread is the execution phase. When the `start()` method is called on a the new instance of a thread, it enters into a runnable state. In the runnable state, thread is ready for execution and is waiting for availability of the processor (CPU time). There are many threads that are ready for execution, they all are waiting in a queue (line).

If all threads have equal priority, a time slot is assigned for each thread execution on the basis of first-come, first-serve manner by CPU. The process of allocating time to threads is known as **time slicing**. A thread can come into runnable state from running, waiting, or new states.

▼ Running



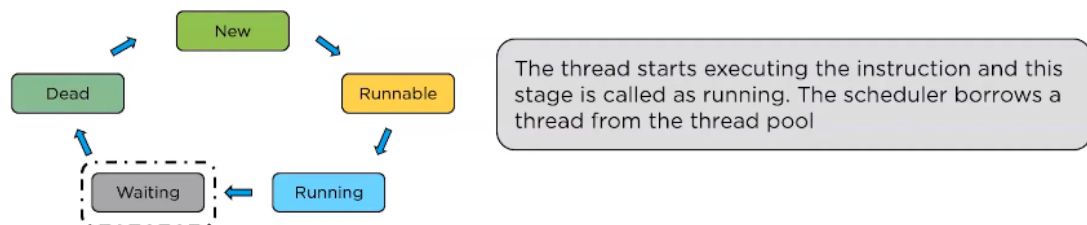
Running means Processor (CPU) has allocated time slot to thread for its execution. When thread scheduler selects a thread from the runnable state for execution, it goes into running state. Look at the above figure.

In running state, processor gives its time to the thread for execution and executes its run method. It is the state where thread performs its actual functions. A thread can come into running state only from runnable state.

A running thread may give up its control in any one of the following situations and can enter into the blocked state.

1. When `sleep()` method is invoked on a thread to sleep for specified time period, the thread is out of queue during this time period. The thread again reenters into the runnable state as soon as this time period is elapsed.
2. When a thread is suspended using `suspend()` method for some time in order to satisfy some conditions. A suspended thread can be revived by using `resume()` method.
3. When `wait()` method is called on a thread to wait for some time. The thread in wait state can be run again using `notify()` or `notifyAll()` method.

▼ Waiting (Blocked)



A thread which is alive but not in runnable or running state will be in waiting(blocked) state. A thread can be in blocked state because of `suspend()`, `sleep()`, `wait()` methods or implicitly by JVM to perform I/O operations.

▼ DEAD(terminated)

A thread reaches the termination state because of the following reasons:

- When a thread has finished its job, then it exists or terminates normally.
- **Abnormal termination:** It occurs when some unusual events such as an `unhandledException` or `segmentation fault`.

A terminated thread means the thread is no more in the system. In other words, the thread is dead, and there is no way one can respawn (active after kill) the dead thread.

▼ Multi-threaded programming

Multithreaded programming is a programming paradigm in which a single process is broken up into two or more threads that can be executed concurrently, in parallel. Each thread represents a separate flow of control, and each thread can run a different part of the program, or the same part of the program with different input data. Multithreaded programming can be used to increase the performance of a program by taking advantage of multiple processors or cores, or to allow a program to perform multiple tasks concurrently, such as downloading data from the internet while also performing calculations. It can also be used to simplify the design of a program by allowing different parts of the program to run concurrently and asynchronously.

Example

```
class WorkerThread extends Thread {
    public void run() {
        // code to be executed by the worker thread
        System.out.println("Worker thread running!!!");
    }
}

public class Main {
    public static void main(String[] args) {
        WorkerThread worker = new WorkerThread();
        worker.start();
    }
}
```

This program creates a new `WorkerThread` and starts it when the `main` method is called. The `run` method of the `WorkerThread` class will be executed by the worker thread when it is started. The output of the program will be:

output

```
Worker thread running!!!
```

▼ Thread Priorities

In Java, each thread has a priority that determines how much CPU time it is allocated. Threads with higher priority will be allocated more CPU time than threads with lower priority. The priority of a thread can be set using the `setPriority`

method of the `Thread` class, and can be any value between `MIN_PRIORITY` (which is 1) and `MAX_PRIORITY` (which is 10). The default priority for a thread is `NORM_PRIORITY`, which is 5.

```
public class Main {
    public static void main(String[] args) {
        Thread thread = new Thread(new Runnable() {
            public void run(){
```

```
        // code to be executed by the worker thread
        System.out.println("Worker thread running");
    }
});
thread.setPriority(Thread.MAX_PRIORITY);
thread.start();
}
}
```

When the program is run, a new thread will be created and the code in the `run` method of the `Runnable` will be executed by the worker thread. The priority of the thread is set to `MAX_PRIORITY` (10), which means that it will be allocated more CPU time than threads with lower priority. However, it is important to note that the actual allocation of CPU time to threads is not guaranteed and can depend on various factors, such as the operating system and the availability of other resources.